HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

CAA - Mini-projet

Ransomware post-quantique

Dylan Oliveira Ramos

04-01-2026

Table des matières

1. Utilisation de l'IA 3
2. Niveau de sécurité choisi 3
3. Description du ransomware 3
4. Communication entre le client et le serveur 4
   4.1. Échange de clés 4
   4.2. Signature de messages 4
5. Clés asymétriques 5
6. Clés symétriques 6
   6.1. Communication Key 6
   6.2. Master Key et File Key 6
   6.3. AES-256-GCM 6
7. Chiffrement des fichiers 8
   7.1. Stockage des métadonnées 9
8. Déchiffrement des fichiers 10
9. Déchiffrement d'un fichier spécifique 11
10. Changement de mot de passe 12

Table des illustrations

Fig. 1 Établissement de la clé symétrique pour la communication sécurisée entre le client et le serveur. 4
Fig. 2 Chiffrement des fichiers et des `File Key` sur le client. 8
Fig. 3 Chiffrement de la `Root Key` sur le client. 8
Fig. 4 Envoi des mots de passe au serveur. 8
Fig. 5 Stockage chez le client et le serveur après le chiffrement des fichiers. 9
Fig. 6 Envoi du `Master Password` au client. 10
Fig. 7 Déchiffrement de la `Root Key` sur le client. 10
Fig. 8 Déchiffrement des `File Key` et des fichiers sur le client. 10
Fig. 9 Envoi du mot de passe du fichier au client. 11
Fig. 10 Déchiffrement d'un fichier spécifique sur le client. 11
Fig. 11 Changement du `Master Password` sur le serveur et mise à jour des métadonnées sur le client. 12

1. Utilisation de l'IA

Ce projet a été réalisé avec l'aide de l'IA, notamment pour la rédaction de la documentation et la génération de certaines parties du code source. Néanmoins, l'architecture cryptographique et le choix des algorithmes ont été conçus par mes soins.

2. Niveau de sécurité choisi

Le ransomware utilise le niveau de sécurité **V**, qui offre une sécurité au moins aussi forte que AES-256. Le respect de ce niveau de sécurité est expliqué tout au long de ce document.

3. Description du ransomware

Le ransomware est composé d'un client (ordinateur de la victime) et d'un serveur (contrôlé par l'attaquant). Pour des raisons de simplicité, les deux entités sont exécutées dans le même programme et les paiements de rançon sont simulés.

Lors du démarrage du programme, les options sont les suivantes :

1. `Encrypt files` : pour chiffrer tous les fichiers d'un dossier choisi.
2. `Pay ransom` : pour payer la rançon et recevoir le `Master Password` permettant de déchiffrer tous les fichiers.
3. `Decrypt one file` : pour payer une plus petite rançon et recevoir le mot de passe permettant de déchiffrer un fichier spécifique.
4. `Change master password` : pour changer le `Master Password` utilisé pour chiffrer/déchiffrer tous les fichiers.

4. Communication entre le client et le serveur

Étant vulnérable aux attaques quantiques, la cryptographie asymétrique classique doit être remplacée par des algorithmes post-quantiques, notamment pour l'échange de clés et la signature de messages.

4.1. Échange de clés

Le mécanisme d'échange de clés post-quantiques **ML-KEM** (Kyber) permet de se protéger contre les attaques « harvest now, decrypt later » en garantissant que même si un attaquant enregistre les communications aujourd'hui, il ne pourra pas les déchiffrer à l'avenir avec un ordinateur quantique. C'est pour cette raison que le ransomware utilise cet algorithme pour établir un secret partagé entre le client et le serveur afin d'en dériver une clé de communication symétrique, qui elle, n'est pas vulnérable aux attaques quantiques.

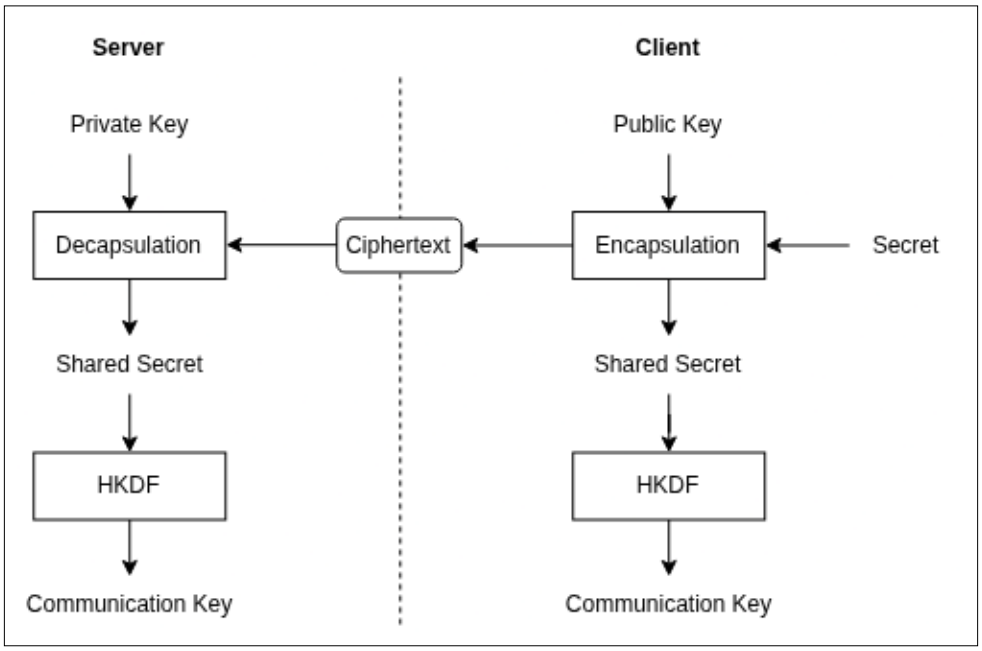

Fig. 1. – Établissement de la clé symétrique pour la communication sécurisée entre le client et le serveur.

4.2. Signature de messages

Pour s'assurer que les mots de passe envoyés par le serveur sont légitimes et n'ont pas été modifiés par un attaquant, le serveur signe chaque message envoyé avec sa clé privée **ML-DSA** (Dilithium) qui est un algorithme de signature post-quantique. Le client vérifie ensuite la signature avec la clé publique du serveur avant d'utiliser les mots de passe reçus.

5. Clés asymétriques

Le programme utilise deux paires de clés asymétriques, le serveur possède les clés privées et le client les clés publiques. La première paire est utilisée pour établir le secret partagé entre le client et le serveur avec l'algorithme **Kyber-1024**. La deuxième paire est utilisée pour signer les messages envoyés par le serveur avec l'algorithme **Dilithium 5**.

Ces deux algorithmes ont été choisis car ils sont post-quantiques, ont un bon compromis taille/sécurité et offrent le niveau de sécurité **V** défini précédemment. La taille des clés de **Kyber-1024** est de 1568 bytes pour la clé publique et 3168 bytes pour la clé privée. La taille des clés de **Dilithium 5** est de 2592 bytes pour la clé publique et 4864 bytes pour la clé privée, les signatures ont une taille de 4595 bytes.

6. Clés symétriques

La cryptographie symétrique n'étant pas vulnérable aux attaques quantiques (à condition que les clés soient suffisamment longues), l'utilisation d'algorithmes tels que **AES-256** reste appropriée pour chiffrer les données du ransomware.

Il y a quatre types de clés symétriques utilisées dans le programme :

- `Communication Key` : clé dérivée du secret partagé avec **HKDF**, utilisée pour chiffrer les communications entre le client et le serveur avec **AES-256-GCM**.
- `Master Key` : clé dérivée d'un mot de passe aléatoire du dictionnaire (`Master Password`) avec **Argon2id**, utilisée pour chiffrer la `Root Key` avec **AES-256-GCM**.
- `Root Key` : clé générée de manière aléatoire, utilisée pour chiffrer les `File Key` avec **AES-256-GCM**.
- `File Key` : clé dérivée d'un mot de passe aléatoire du dictionnaire avec **Argon2id**, unique pour chaque fichier, utilisée pour chiffrer le fichier avec **AES-256-GCM**.

6.1. Communication Key

Pour obtenir la `Communication Key`, l'algorithme **HKDF** est utilisé car il est conçu pour dériver des clés à partir d'entrées avec une bonne entropie, ce qui est le cas des secrets partagés.

Cette dérivation s'effectue avec les paramètres suivants :

- Algorithme de hachage : SHA-256.
- Taille de la clé dérivée : 32 bytes (pour être compatible avec AES-256).
- Sel : aucun (RFC 5869).

6.2. Master Key et File Key

Une des contraintes du projet est de devoir utiliser des mots de passe pour déchiffrer les fichiers, pour cette raison, l'algorithme **Argon2id** est utilisé pour dériver la `Master Key` et les `File Key`. Cet algorithme est adapté à ce cas d'utilisation car il est conçu pour dériver des clés à partir d'entrées à entropie faible, comme des mots de passe. De plus, il est résistant aux attaques par force brute, ce qui permet de compliquer la tâche d'un attaquant qui tenterait de deviner les mots de passe pour déchiffrer les fichiers.

Cette dérivation s'effectue avec les paramètres suivants (paramètres par défaut, à adapter selon la machine) :

- Taille du sel : 16 bytes (taille recommandée).
- Taille de la clé dérivée : 32 bytes (pour être compatible avec AES-256).
- Nombre d'itérations : 1.
- Degré de parallélisme : 4.
- Coût en mémoire : 65536 KB.

6.3. AES-256-GCM

Le chiffrement symétrique des différentes données du ransomware est effectué avec l'algorithme **AES-256-GCM**. Cet algorithme a été choisi car il est performant, sécurisé et largement utilisé dans l'industrie. La taille de clé de 32 bytes (256 bits) a été choisie pour correspondre au niveau de sécurité **V** défini précédemment.

Les paramètres utilisés sont les suivants :

- Taille de la clé : 32 bytes.
- Taille du nonce : 12 bytes (taille recommandée).
- Taille du tag : 16 bytes.

Ces paramètres permettent théoriquement de chiffrer des fichiers d'une taille maximale d'environ **68 GiB**. À noter que pour des raisons de simplicité, le programme charge l'intégralité des fichiers en mémoire avant de les chiffrer/déchiffrer, ce qui peut poser problème pour des fichiers très volumineux.

7. Chiffrement des fichiers

Chaque fichier est chiffré avec sa `File Key` respective. Cette `File Key` est ensuite chiffrée avec la `Root Key`, qui est elle-même chiffrée avec la `Master Key`.

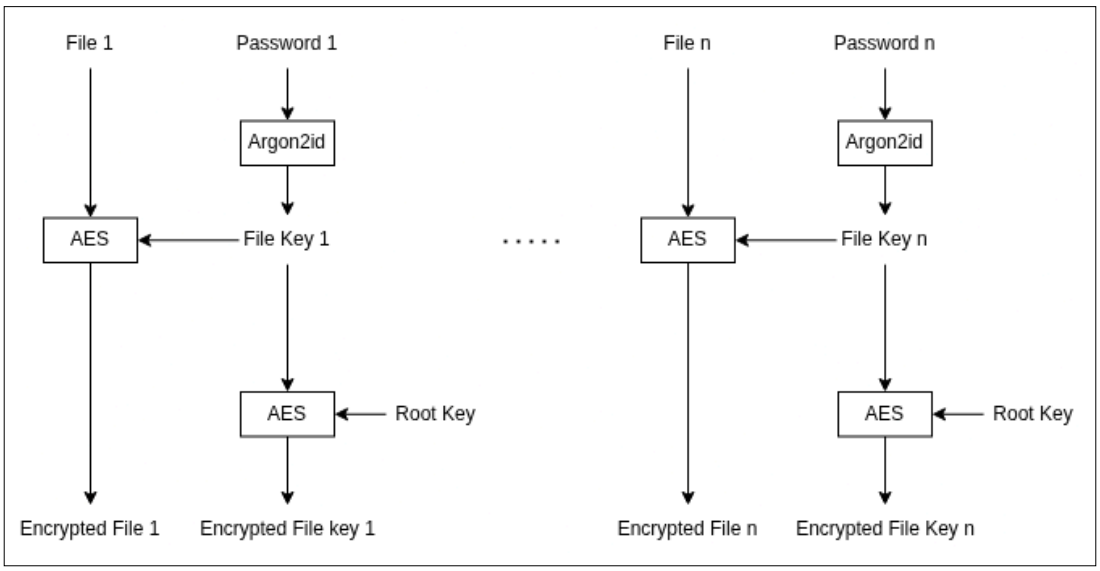

Fig. 2. – Chiffrement des fichiers et des `File Key` sur le client.

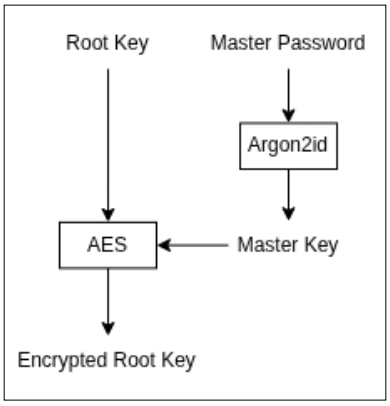

Fig. 3. – Chiffrement de la `Root Key` sur le client.

À la fin du chiffrement, tous les mots de passe utilisés pour dériver les clés sont envoyés au serveur et supprimés du client.

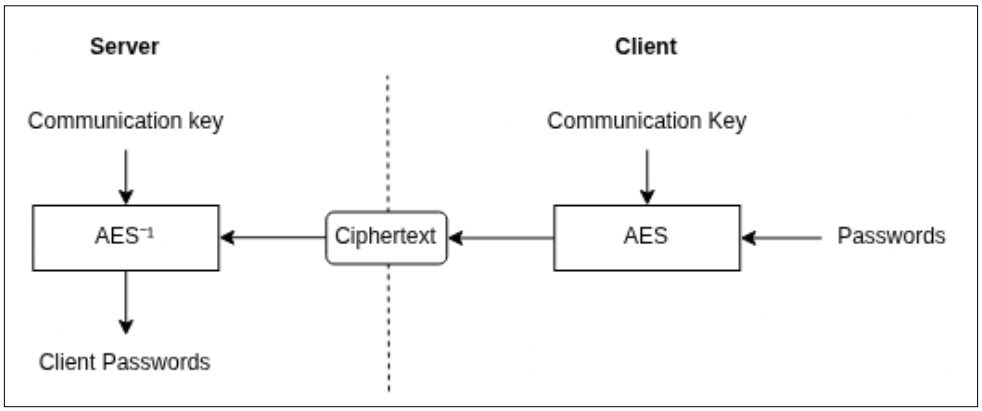

Fig. 4. – Envoi des mots de passe au serveur.

7.1. Stockage des métadonnées

Pour que les fichiers puissent être déchiffrés sans avoir à les envoyer au serveur, il faut stocker certaines métadonnées sur le client. Ainsi, lors du chiffrement d'un fichier, son contenu est remplacé par le contenu concaténé suivant :

File ID	Password Salt	File Key IV	File Key Tag	File Key Ciphertext	File IV	File Tag	File Ciphertext

Le stockage de ces données permet de déchiffrer le fichier ultérieurement de deux manières :

1. Avec le mot de passe du fichier (utilisé avec le sel stocké pour dériver la `File Key`), dans le cas où l'utilisateur paie la rançon pour déchiffrer un seul fichier.
2. Avec la `Root Key` (utilisée pour déchiffrer la `File Key`), dans le cas où l'utilisateur paie la rançon pour déchiffrer tous les fichiers.

Lorsque tous les fichiers ont été chiffrés, un fichier de métadonnées (ayant pour nom un UUID) est créé à la racine du dossier, contenant les données concaténées suivantes :

File ID	Master Password Salt	Root Key IV	Root Key Tag	Root Key Ciphertext

Le stockage de ces données permet de déchiffrer la `Root Key` ultérieurement avec la `Master Key` (dérivée du `Master Password` et du sel). Ce fichier de métadonnées est également nécessaire pour changer le `Master Password` lorsque le serveur en fait la demande.

Note : Le `File ID` 0 est réservé pour le fichier de métadonnées à la racine.

La Fig. 5 ci-dessous illustre ce que stockent le client et le serveur une fois le chiffrement terminé.

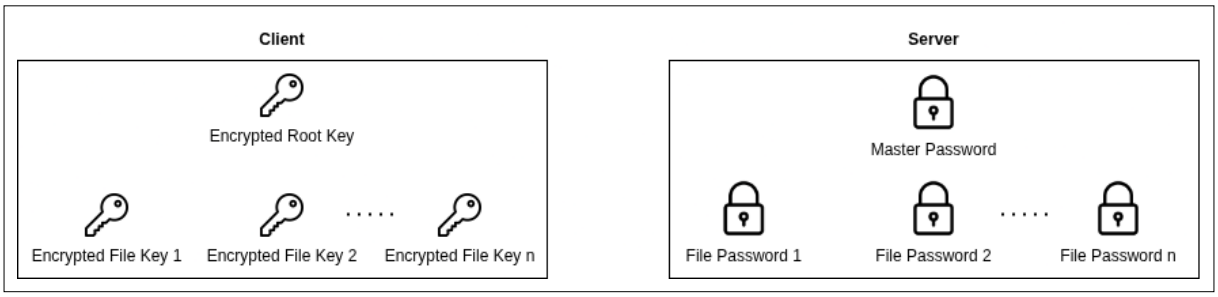

Fig. 5. – Stockage chez le client et le serveur après le chiffrement des fichiers.

8. Déchiffrement des fichiers

Pour déchiffrer tous les fichiers, il faut connaître le `Master Password`, qui est envoyé par le serveur une fois la rançon payée. Celui-ci permet de dériver la `Master Key`, qui permet de déchiffrer la `Root Key`, qui permet de déchiffrer les `File Key` et enfin les fichiers.

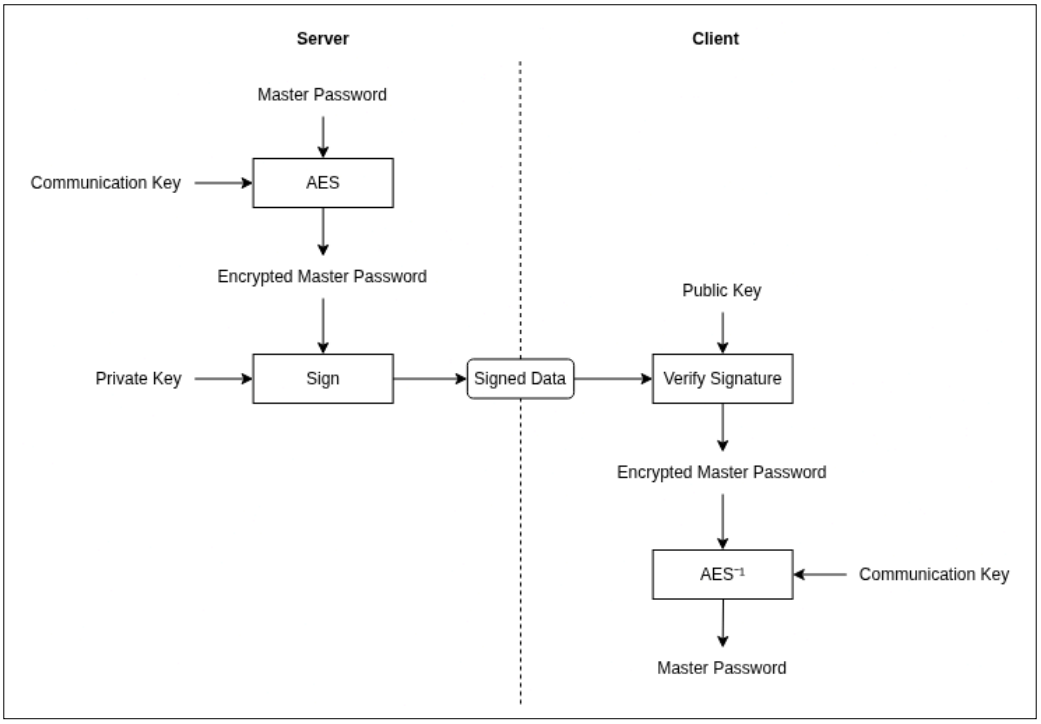

Fig. 6. – Envoi du `Master Password` au client.

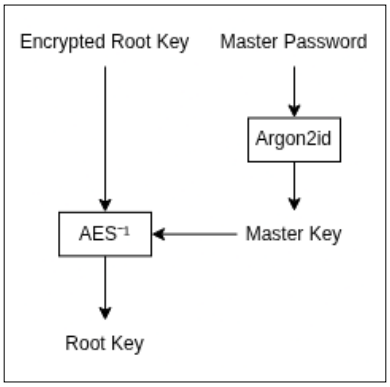

Fig. 7. – Déchiffrement de la `Root Key` sur le client.

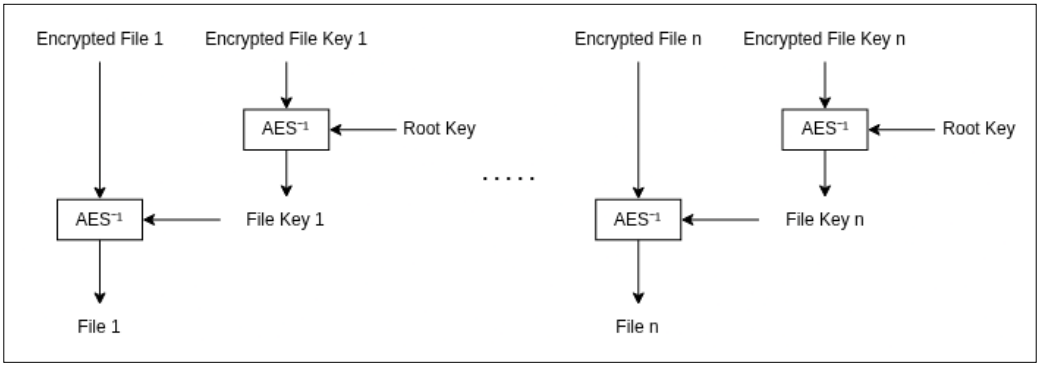

Fig. 8. – Déchiffrement des `File Key` et des fichiers sur le client.

9. Déchiffrement d'un fichier spécifique

Pour déchiffrer un fichier spécifique, il faut connaître le mot de passe du fichier, qui est envoyé par le serveur une fois la rançon payée. Celui-ci permet de dériver la `File Key`, qui permet de déchiffrer le fichier. Pour cela, le client récupère l'identifiant du fichier choisi (stocké dans le fichier lui-même) puis l'envoie au serveur pour obtenir le mot de passe correspondant.

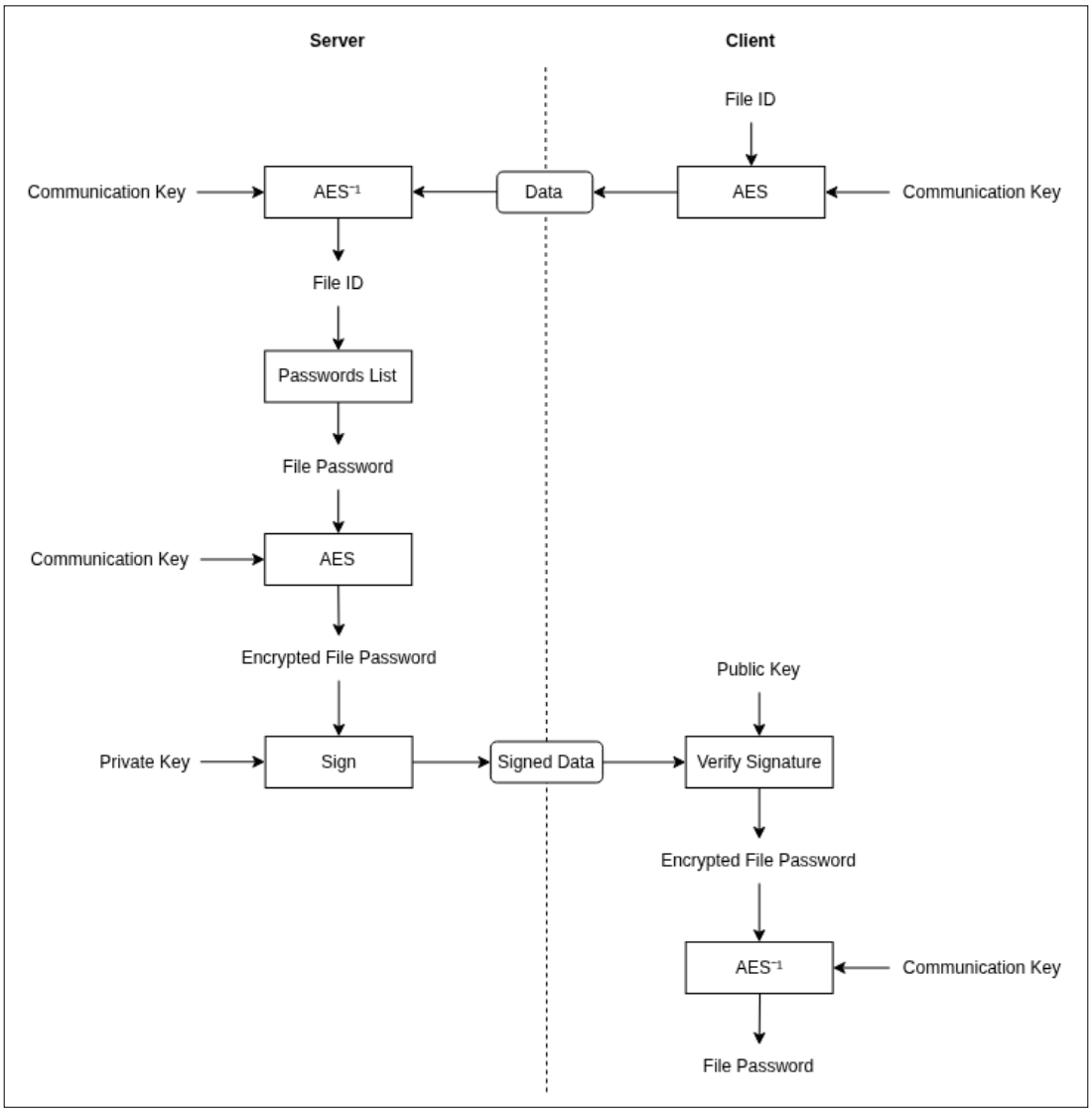

Fig. 9. – Envoi du mot de passe du fichier au client.

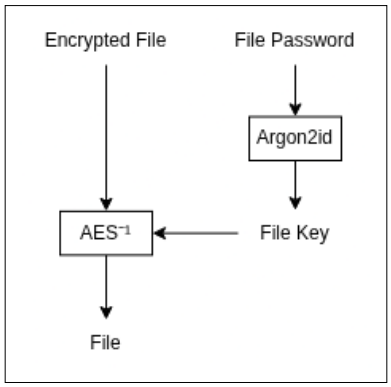

Fig. 10. – Déchiffrement d'un fichier spécifique sur le client.

10. Changement de mot de passe

Il est possible de changer le `Master Password` après le chiffrement des fichiers sans avoir à rechiffrer tous les fichiers. En effet, le `Master Password` n'est utilisé que pour chiffrer la `Root Key`. Ainsi, pour changer le mot de passe, il suffit de déchiffrer la `Root Key` en utilisant l'ancien mot de passe, puis de la rechiffrer avec le nouveau mot de passe.

Lors de la demande de changement, le client envoie les métadonnées (fichier créé à la racine du dossier) ainsi qu'un nouveau `Master Password` au serveur. Étant donné que le serveur connait le `Master Password` initial, il peut déchiffrer la `Root Key` et la rechiffrer avec le nouveau mot de passe. Ensuite, il renvoie les nouvelles métadonnées au client pour qu'il puisse les stocker.

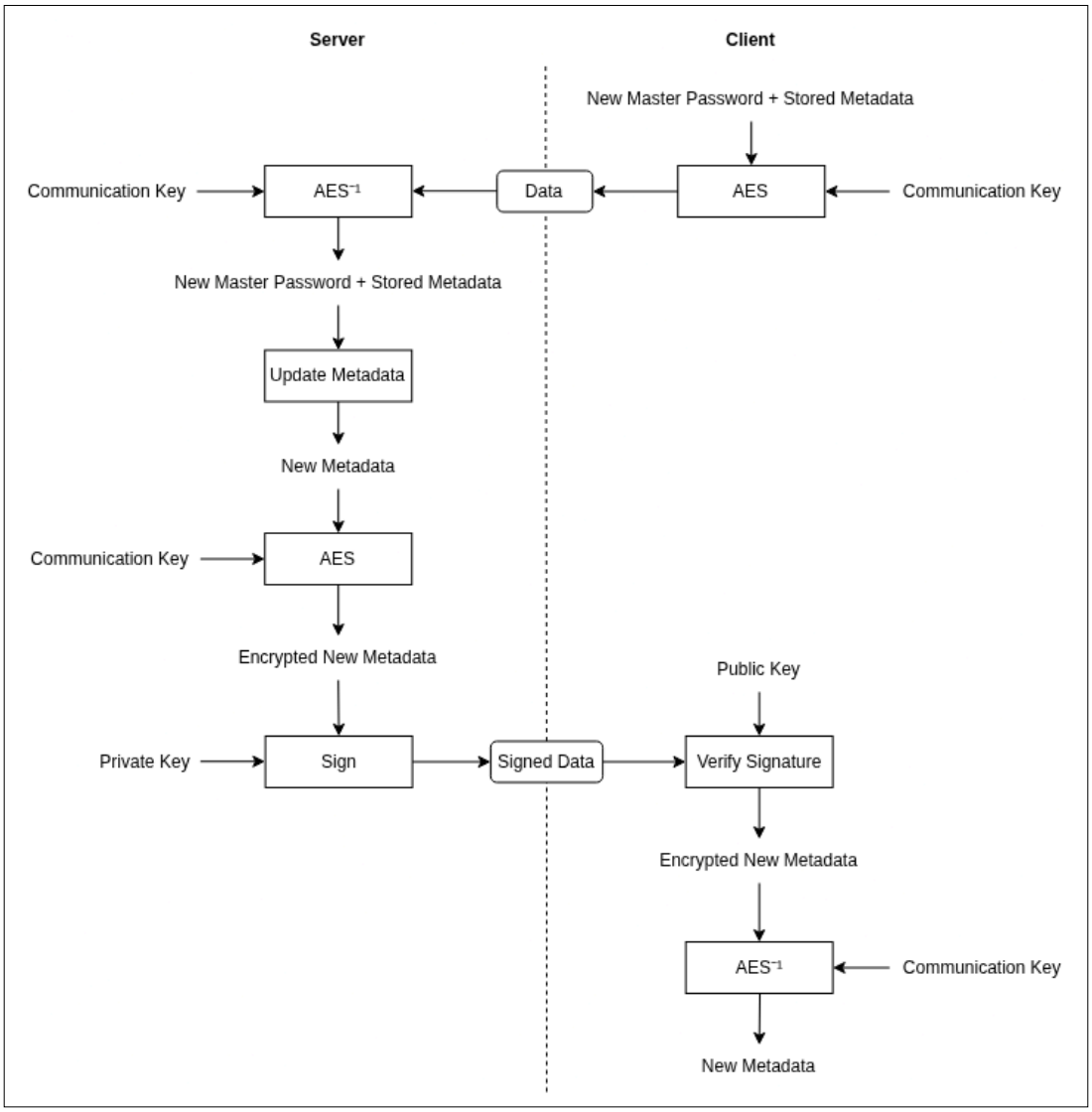

Fig. 11. – Changement du `Master Password` sur le serveur et mise à jour des métadonnées sur le client.